

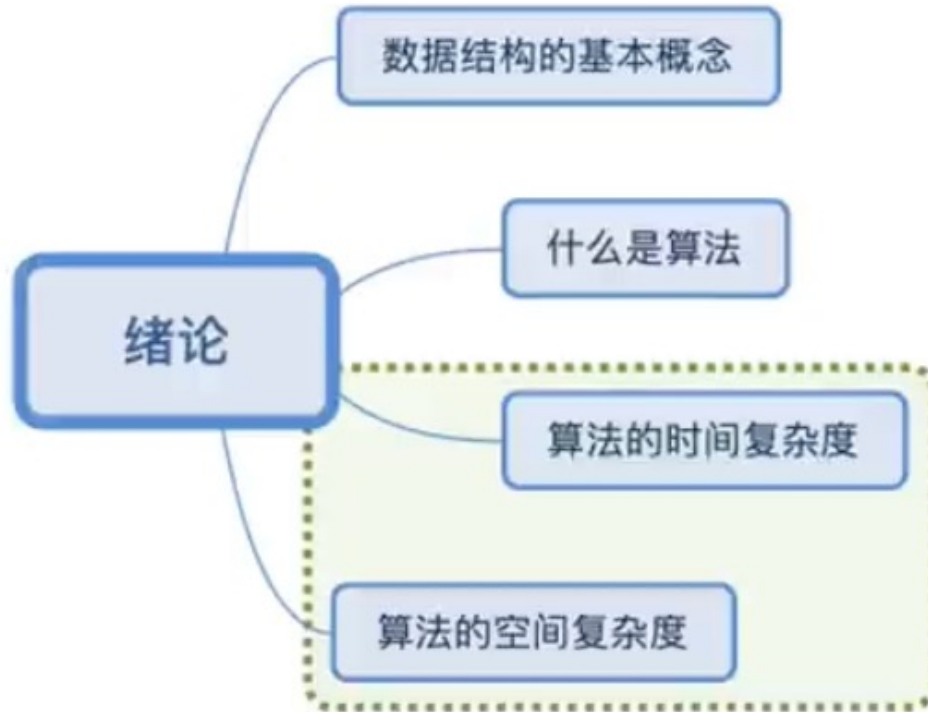


数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



1.4节

1.5节



§1.4 算法定义



Nicklaus Wirth (1984年Turing Award)

“凭借一句话”获得图灵奖的Pascal之父



算法 + 数据结构 = 程序
(Algorithms + Data Structures = Programs)

处理问题的策略

问题的数学模型

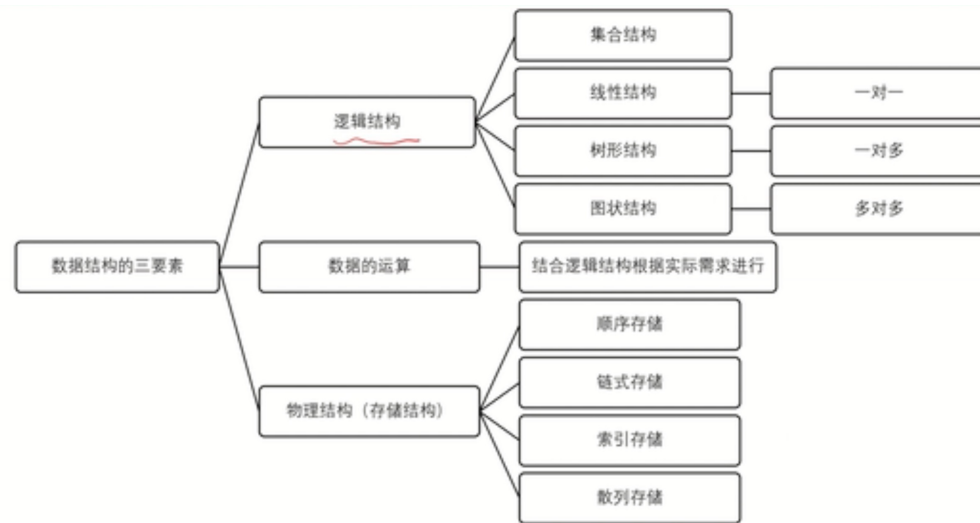
为计算机处理问题编制的一组指令集



数据结构



- 逻辑结构（线性、非线性）
 - 线性结构：一维数组、线性表、栈、队列、双队列、串
 - 非线性结构：分为多维数组、集合、树、图、散列表
- 运算结构（分支、循环）
 - 分支结构：if、switch、三目运算符
 - 循环结构：for、while、do...while
- 存储结构
 - 数组：随机查询；
 - 链表：增删很简单



①ArrayList集合

本质是一种可以扩容的数组；增删慢；查询快。数组就是随机存储。

②LinkedList集合

本质是一个链表；增删快；查询慢。

在使用LinkedList集合时，最好使用LinkedList创建对象；不要用多态，因为多态无法使用LinkedList的特有功能。



算法



算法是指一系列解决问题的清晰指令，用系统的方法描述解决问题的策略机制。也就是说，能够对一定规范的输入，在有限时间内获得所要求的输出。

通俗说就是**解决问题的方法和步骤**。

问题：从上海去到北京。

其中的算法：坐汽车、坐火车、坐飞机，或者徒步。

问题：喝茶。

其中的算法：先找到茶叶，再烧一壶开水，然后将茶叶放到杯子里，将开水倒入杯中，等茶叶泡好。

问题：开车。

其中的算法：首先要打开车门，驾驶员坐好，插上车钥匙，发动汽车。



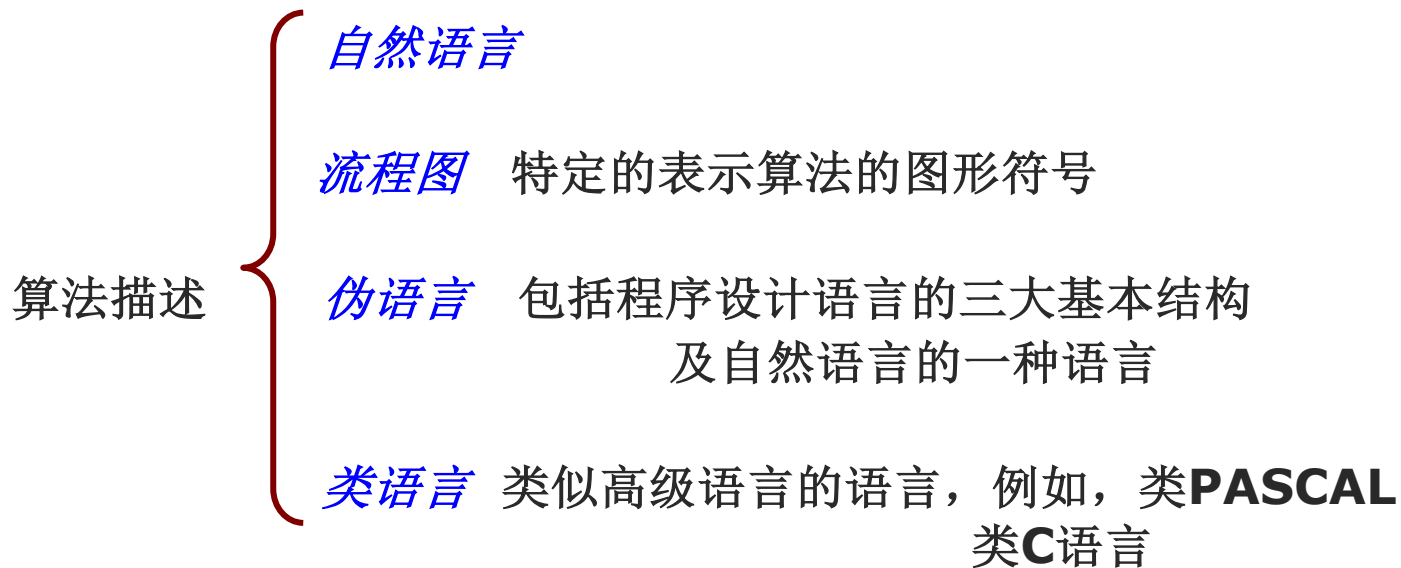
Problem Solving: Main Steps

1. Problem definition
2. Algorithm design / Algorithm specification
3. Algorithm analysis
4. Implementation
5. Testing
6. [Maintenance]



算法的描述

常用的算法描述方式:





用辗转相除法求两数的最大公约数

- (1) 用 m 除以 n ，令所得的余数为 r
- (2) 若 $r=0$ ，则输出 n ，算法结束，否则继续 (3)
- (3) 令 $m=n, n=r$ ，并返回步骤 (1)

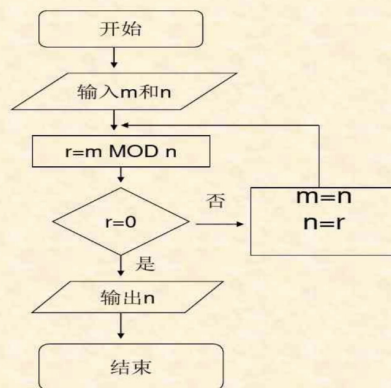
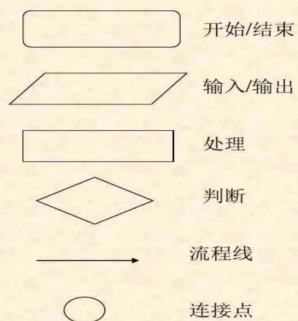
例，求112和68的最大公约数

$m=112$	$n=68$	$r=44$
$m=68$	$n=44$	$r=24$
$m=44$	$n=24$	$r=20$
$m=24$	$n=20$	$r=4$
$m=20$	$n=4$	$r=0$

输出 n 的值

m	n	余数r

流程图



```
INPUT m,n
r=m MOD n
DO WHILE r<>0
m=n
n=r
r=m MOD n
LOOP
PRINT n
```



一个算法必须满足以下五个重要特性：

- **有输入** 有0个或多个输入
- **有输出** 有一个或多个输出(处理结果)
- **确定性** 每步定义都是确切、无歧义的
- **有穷性** 算法应在执行有穷步后结束
- **能行性** 每一条运算应足够基本



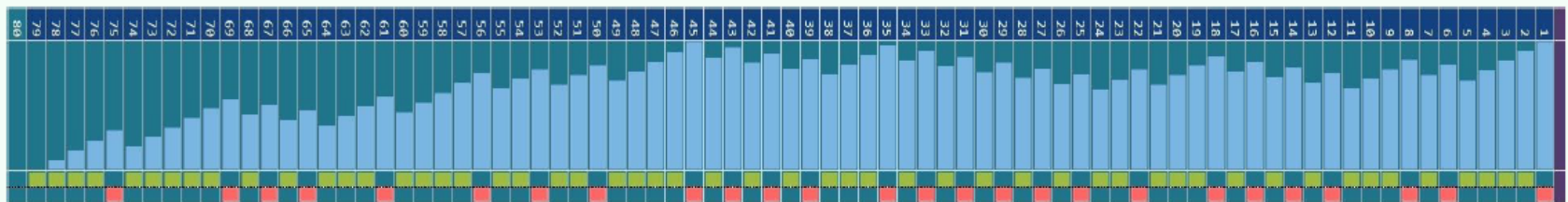
```
void selectSort (int a[ ], const int n ) {  
    //按非递减顺序对n个关键码a[0] ~ a[n-1]排序  
    for ( int i = n - 1; i > 0; i-- ) {  
        int j = MaxKey (a, 0, i);  
        if ( j != i ) swap (j, i);  
    }  
}  
  
int MaxKey (int a[ ], const int low, const int high) {  
    //查找数组a[low] ~ a[high]中的最大值, 函数返回其位置  
    int max = low;  
    for (int k = low+1, k <= high, k++)  
        if ( a[max] < a[k] )  
            max = k;  
  
    return max;  
}
```





有穷性：程序~算法

```
int hailstone( int n ) {  
    int length = 1;  
    while ( 1 < n ) { n % 2 ? n = 3*n + 1 : n /= 2; length++; }  
    return length;  
} //对于任意的n, 总有|Hailstone(n)| <  $\infty$  ?
```



❖ Erdos: Mathematics is **NOT** yet ready for such problems



§1.5 算法性能分析与度量



1.5.1 算法的性能标准

- 正确性
- 可使用性
- 可读性
- 效率
- 健壮性：输入非法数据时，能适当做出反应或进行处理
- 简单性



● 可读性

```
#include<stdio.h>
typedef unsigned int _; _ d,b,
#define i(I1,I1,LI)if(I1){LI;}else{I1;}
I[256], n,y,a,r,u,k,o
,L,l[ 256],O,K[
/*+*/ #define\
q(g) g char\
*c, *Q,c[\
]= "KfW"\
"" "Ww"\
/* % 'UU!\
*/ % NYA!\
"Z)"\
".s"\
"Ro"\
"di"\
"ef"\
"e"\
"n"\
"u"\
"o"\
")"\
"#"\
"-"
/*z;
) ]+u
(u>>
;256
^=K[
main
"rb"
=O=
for
!=
=s
(p
^=
O=
2
;

";fRo?JtJaV<x4?*R?&JV1"
" {Fy]2_ ;khB1xQ5oxm~mS@B| (pa>oRU"
"nB)h@o?)d.X)NSTIUCz7@%,"s[]={c,"#en"
"f/*}||1;\n _DATA_ \40*/\n\n#endif\40q\n#d"
"ine\x20q\n#include<stdio.h>\ntypedef\40unsign"
"d\x20int\x20_;\x20K[]={\n#include\40_ FILE_ \n#u"
"def q","0",L,O,I[256],I[256],n,y,a,r,u,k,o;"#g"char"
"s,s[]=""",c,c,"";int main() {X();for(s=s+*K;*s>37;) {for"
"=0;o<5;o++)r=x*85+(83+*s++)%89;r","^=*x();for(o=0;o<4;o++"
"[s[o++]=r%255;r>=8;)}return!fwrite(s,O-*s%5,1,stdout);}\n"
"endif",c),s[256]="#ifdef/*\n'true'\40or\40q{\nexec\40head\40"
"8\40$0\n};for(open$0,$0;<$0>;print\40if$F){f|=/^$/; }q{"/q",/*
256;q( _*x() {if(!L--){y+=++a;for(o=0;o<256;y=1[o++]=I[255&(k>>10
)}{n^=(o&1)?n>>((o& 2)?16:6):n<<((o&2)?2:13);u=I[o];k=I[o]=I[255&
2)]+(n+=I[(o+128)& 255]) +y;}L=255;}return&l[L];}_*X() {for(O=0
>O;I[O++]=0);for(O = 0; sizeof(K)/sizeof( _)> O;O++)I[O&255]
O];for(n=y=a=L=O=0;O<1<<24;++ O)x( );r=O=0x0; return&O;)}int/*^*/
(int p,char**P){FILE* Z=fopen(p> (+ 1)?P[01 ] :"/dev/urandom",
);i(,2,O=fread(K,256 ,4,Z);/*P
0;O<256;K[O++]=0)*K+= 86;for(O
(C=Q=s;x-8;){i(*C++=34, (x-4&&r
32||r-3,i(C+=sprintf(( C),
; i(d=02,b-12,;),b%6,;),r-1
=0=0,b,for(d=0=0;O<04;O++)d
*x();for(p=5;p;c[--p]=O<32?O+
5)i(d=0,b<4,c[O++]=b?b-1?b-2?36:
|d,C|=S+6,*C++=( *x() %34)+93;p--),r
z-63||C [-1]-63||C>S+76,;)*C++=z)}i(
,r<3|| x>5,d=1;i(,r-1, *C=0)=S) i(,
34,r <4||r>5|| C<S+ 78,; )i (C++=0;d=1; C=S ,r<3 || r>
5 ||C<S+ 79,; )i(,d, puts (
S); d=0 )} return 0;}
```

国际C语言混乱代码大赛（IOCCC, The International Obfuscated C Code Contest）



- 算法的效率包括**时间代价**和**空间代价**。
 - 前者指的是**算法执行时间**;
 - 后者指的是算法执行过程中**所需的最大存储空间**。
- 两者都与问题的规模有关。



算法的基本概念

什么是算法

程序 = 数据结构 + 算法

数据结构是要处理的信息

算法是处理信息的步骤

算法的五个特性

有穷性

有穷时间内能执行完

算法是有穷的

程序可以是无穷的

确定性

相同输入只会产生相同输出

可行性

可以用已有的基本操作实现算法

输入

丢给算法处理的数据

输出

算法处理的结果

正确性

能正确解决问题

可读性

对算法的描述要让其他人也看得懂

健壮性

算法能处理一些异常状况

"好"算法的特质

高效率与低存储量需求

即算法执行省时、省内存

时间复杂度低、空间复杂度低



算法效率的衡量方法:

- 后期测试
- 事前估计



1.5.2 算法的后期测试

在算法中的某些部位插桩时间函数 *time* ()
测定算法完成某一功能所花费的时间

存在什么问题？

- 和机器性能有关，如：超级计算机/单片机
- 和编程语言有关，越高级的语言执行效率越低
- 和编译程序产生的机器指令质量有关
- 有些算法是不能事后再统计的，如：导弹控制算法



1.5.3 算法的事前估计

- 算法的复杂性度量属于事前估计
 - 空间复杂度
算法执行时所占用的存储空间
 - 时间复杂度
算法执行时所耗费的时间



时间复杂度

- **编译时间**
- **运行时间**
 - **程序步：语法（义）上有意义的一段指令**

例如：

注释：程序步数为0

声明语句：程序步数为0

表达式、赋值语句：程序步数为1

循环语句：循环控制语句每次执行的程序步数为1

.....



- 程序步确定方法
 - ◆ 插入计数全局变量 *count*

例：以迭代方式求累加和的函数

```
行 float sum ( float a[ ], const int n )  
1  {  
2    float s = 0.0;  
3    for ( int i = 0; i < n; i++ )  
4      s += a[i];  
5    return s;  
6  }
```



在求累加和程序中加入 $count$ 语句

```
float sum ( float a[ ], const int n )      {  
    float s = 0.0;  
     $count++$ ;           // $count$ 统计执行语句条数  
    for ( int i = 0; i < n; i++ ) {  
         $count+=2$ ;      //针对for语句  
        s += a[i];  
         $count++$ ;      //针对赋值语句  
    }  
     $count+=2$ ;          //针对for的最后一次  
     $count++$ ;           //针对return语句  
    return s;  
}      执行结束得 程序步数  $count = 3 * n + 4$ 
```



注意:

一个语句本身的程序步数可能不等于该语句一次执行所具有的程序步数。

例如：赋值语句

$x = \text{sum}(R, n);$

本身的程序步数为1;

一次执行对函数 $\text{sum}(R, n)$ 的调用需要的程序步数为
 $3*n+4$;

一次执行的程序步数为

$$1+3*n+4 = 3*n+5$$

确定程序的准确程序步数十分困难。程序步数本身也不是一个精确的概念，不能确切反映运行时间。



1.5.4 算法的渐近分析

- 一个算法的“运行时间”通常是**随问题规模的增长而增长**,它是问题规模(用正整数 n 表示)的函数。
- 统计算法中的**关键操作**,以**关键操作重复执行的次数**作为算法的时间度量。
- 算法中关键操作重复执行的次数是规模 n 的某个函数 $T(n)$



例1: $s = 0;$

例2: $s = 0;$ //a[]中各行进行累加

```
for ( int  $i = 0$ ;  $i < n$ ;  $i++$  )  
     $s += a[i];$ 
```

例3:

```
for ( int  $i = 0$ ;  $i < n$ ;  $i++$  ) { //x[ ][ ]中各行数据累加  
     $sum[i] = 0.0;$   
    for ( int  $j=0$ ;  $j<n$ ;  $j++$  )  
         $sum[i] += x[i][j];$   
}
```

各个程序段中的关键操作重复执行的次数?



- 要精确地计算 $T(n)$ 通常较困难
- 引入**渐近时间复杂度**——在数量级上估计一个算法的执行时间，也能够达到分析算法的目的。（**观察算法随 n 增长的趋势**）

(大O记号): $T(n) = O(f(n))$

大O表示法：当且仅当存在正整数 c 和 n_0 ，使得 $T(n) \leq c \cdot f(n)$ 对所有 $n \geq n_0$ 成立，则称该算法的时间增长率在 $O(f(n))$ 中，记为 $T(n) = O(f(n))$ 。

例：一个程序的实际执行时间为

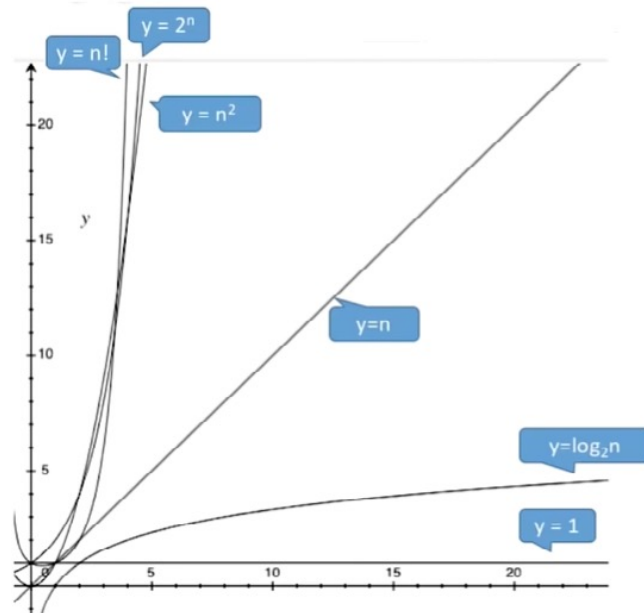
$T(n) = 2.7n^3 + 3.8n^2 + 5.3$ 则 $T(n) = O(n^3)$



- 使用大 O 记号表示的算法的时间复杂度，称为算法的渐近时间复杂度（Asymptotic Complexity）
- 通常用 $O(1)$ 表示常数计算时间。常见的渐近时间复杂度有：

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(3^n) < O(n!)$$

常对幂指阶





大O表示法的几条规则:

如果 $T1(n) = O(f(n))$, $T2(m) = O(g(m))$

- **加法规则** 针对并列程序段

$$T(n, m) = T1(n) + T2(m) = O(\max(f(n), g(m)))$$

- **乘法规则** 针对多层嵌套循环

$$T(n, m) = T1(n) * T2(m) = O(f(n) * g(m))$$

- **乘法规则中的常数无关项** $O(c) \rightarrow O(1)$

$$T(n) = O(c * f(n)) = O(f(n))$$

- **$O(1)$ 表示常数计算时间**



例1：两个并列循环

```
void example (float x[ ][ ], int m, int n, int k) {  
    float sum [ ];  
    for ( int i = 0; i < m; i++ ) { //x[ ][ ]中各行  
        sum[i] = 0.0;                //数据累加  
        for ( int j=0; j<n; j++ )  
            sum[i] += x[i][j];  
    }  
    for ( i = 0; i < m; i++ ) //打印各行数据和  
        cout << "Line " << i <<  
            " : " << sum [i] << endl;  
}
```

渐近时间复杂度为 $O(\max(m*n, m))$



例2：冒泡排序

```
template <class Type>
void dataList<Type>::bubbleSort ( ) {
//对表 Element[0] 到 Element[ArraySize-1]
//逐趟进行比较, ArraySize 是表当前长度
    int i = 1; int exchange = 1;
        //当exchange为0则停止排序
    while ( i < ArraySize && exchange ) {
        BubbleExchange ( i, exchange );
        i++;
    } //一趟比较
}
```



```
template <class Type>
void dataList<Type>::
BubbleExchange(const int i, int & exchange ){
    exchange = 0;           //假定元素未交换
    for ( int j = ArraySize-1; j>=i; j-- )
        if ( Element[j-1] > Element[j] ) {
            Swap ( j -1, j );    //发生逆序
                                //交换Element[j-1]与Element[j]
            exchange = 1;       //做 “发生了交换” 标志
        }
    }
```



时间复杂度

如何计算

① 找到一个基本操作（最深层循环）

② 分析该基本操作的执行次数 x 与问题规模 n 的关系 $x=f(n)$

③ x 的数量级 $O(x)$ 就是算法时间复杂度 $T(n)$

加法规则: $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$

常用技巧

乘法规则: $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$

“常对幂指阶”

$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$

三种复杂度

最坏时间复杂度: 考虑输入数据“最坏”的情况

平均时间复杂度: 考虑所有输入数据都等概率出现的情况

最好时间复杂度: 考虑输入数据“最好”的情况



- 1.找出基本操作：通常是循环内最耗时的操作。
- 2.找到问题规模 n ：通常是数组长度、输入值的大小等。
- 3.计算执行次数 $T(n)$ ：分析循环嵌套，计算基本操作的总次数。
- 4.简化成大O表示：忽略常数项、系数和低次项，只保留最高次项。
口诀：“数循环，看嵌套，取最高”。



渐近空间复杂度

$$S(n) = O(f(n))$$

$S(n)$ 为算法的空间复杂度函数

- 反映的是当问题规模 n 充分大时，需要的存储空间将如何随之变化。

1. 忽略输入数据占用的空间。
2. 查看算法声明的变量、数据结构和申请的内存：
 1. 如果只用了固定数量的变量（如 `int i, j, temp`），则是 $O(1)$ 。
 2. 如果申请了和输入规模 n 相关大小的数组、链表等，则是 $O(n)$ 、 $O(n^2)$ 等。
3. 如果是递归算法，空间复杂度通常与最大递归深度成正比。
4. 取最大值：如果算法中有多个部分都申请了空间，总空间复杂度取最大值（增长最快的那个）。
口诀：“看申请，找 n ，递归就看深度”。



空间复杂度

如何计算

普通程序

① 找到所占空间大小与问题规模相关的变量

② 分析所占空间 x 与问题规模 n 的关系 $x=f(n)$

③ x 的数量级 $O(x)$ 就是算法空间复杂度 $S(n)$

递归程序

① 找到递归调用的深度 x 与问题规模 n 的关系 $x=f(n)$

② x 的数量级 $O(x)$ 就是算法空间复杂度 $S(n)$

注：有的算法各层函数所需存储空间不同，分析方法略有区别

常用技巧

加法规则： $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$

乘法规则： $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$

“常对幂指阶”

$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$



书面作业提交时间：9月9日上课前
提交到仙1-201讲台

第1章：

- (1) 第38页，1.7：(1), (4)
- (2) 第38页，1.10：(2)
- (3) 第39页，1.11：(1)
- (4) 第40页，1.13：(7)